Segon control INEP, 16-Gener-2024 Temps de resolució: 2 hora 45 minuts

Entregueu els dos problemes en fulls separats. Els dos problemes són independents l'un de l'altre

L'acadèmia Bits4All, que coneixem tant bé del primer parcial, ha adoptat una versió simplificada de l'especificació que li vam proposar. Podeu veure en la pàgina adjunta el model conceptual de dades, que expliquem tot seguit. De cada curs, Bits4All va obrint noves edicions quan ho considera convenient. Els estudiants es poden preinscriure de diversos cursos. Les preinscripcions es processen al moment i o bé s'accepten, assignant-li un tutor a l'estudiant i matriculant l'estudiant en una edició del curs, o bé es rebutgen, enregistrant-ne el motiu.

Requisits de dades:

- Cursos. Dels cursos se'n sap el seu nom, el codi (que els identifica) i el número d'hores. L'oferta de cursos es crea en posar en marxa el sistema (i no serà buida).
- Edicions de cursos. Cada edició té una data d'inici, una data de fi i una capacitat (número d'estudiants que s'hi poden matricular). Dues edicions d'un mateix curs no es poden solapar en el temps. (Nota: no cal que tracteu altres possibles restriccions de dates.)
- Estudiants. Dels estudiants se'n sap el seu dni (que els identifica), nom i titulació universitària superior. Els estudiants no poden renunciar a preinscripcions ni a matrícules.

Problema 1: Disseny amb Domain Model i Data Mapper [6 punts].
Siguin els casos d'ús essencials següents descrits de forma abreujada i només en el curs rellevant d'esdeveniments, el model del comportament dels quals es mostra en el full adjunt:

CU1. *Nova Edició*. El Cap d'Estudis demana l'alta d'una nova edició d'un curs existent. El Sistema enregistra la nova edició i a més retorna el nombre de llocs lliures en totes les edicions del curs, incloent-ne els llocs de l'edició tot just creada

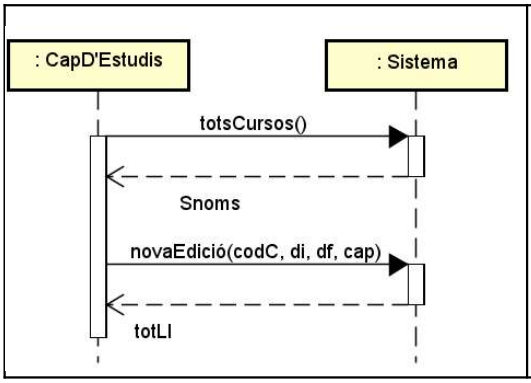
CU2. *Expedient Estudiant*. El Cap d'Estudis demana l'expedient d'un estudiant donat, i el sistema el retorna, segons els detalls donats en el contracte de "expedient" (v. full adjunt).

Apartat 1 [0.75 punts]. Adapteu el model del comportament de **CU1** a un disseny d'interfície en el què, en començar el cas d'ús, mostra per pantalla en un desplegable els codis de tots els cursos existents i aleshores el Cap d'Estudis n'elegeix un i escriu al mateix temps les dues dates i la capacitat, de manera que el sistema avorta el cas d'ús si la data final és menor que la data inicial o la capacitat no és estrictament positiva.

Apartat 2 [1.25 punts]. Considerant el model resultat de l'Apartat 1 per a **CU1**, i assumint que el model del comportament de **CU2** després de fer el disseny de la interfície d'usuari no canvia, declareu el contracte de les operacions de la capa de domini corresponents als dos casos d'ús **CU1** i **CU2**, amb especial cura del tractament de tots els errors.

Apartat 3 [4 punts]. Feu el diagrama de classes de disseny de la capa de domini i dissenyeu el diagrama de seqüència de totes les operacions que us hagin sortit a l'Apartat 2, mostrant clarament el flux des de la capa de presentació fins a la capa de dades. Useu controlador façana a la capa de presentació i transacció a la capa de domini. Escriviu els contractes de les operacions noves que introduïu a la capa de domini (tret de creadores, getters/setters i les del controlador transacció). Suposeu que l'atribut derivat es calcula. No oblideu de mostrar les navegabilitats finals de les associacions en el diagrama de classes de disseny. **Ajuts notacionals**: Podeu usar: **(1)** L'operació "solapa(di1, df1, di2, df2): Bool" que mira si els intervals de temps [d1,df1] i [d2, f2] se solapen. **(2)** L'operació "mostra()" sobre conjunts tals que "s.mostra()" mostra el contingut de s per la pantalla. **(3)** Podeu dir "segons l'especificació donada a l'enunciat" quan us convingui per no haver de repetir totes les condicions de retorn de l'operació "expedient".

1.1



operació totsCursos(): Conjunt(codC: String)
retorna el conjunt dels codis dels cursos existents

operació novaEdició(codC: String; di, df: Data; cap: Integer): Int.
pre existeix Curs c amb c.codi = codC
post crea una Edició e = (di, df, cap) i l'associa amb c
retorna el nombre de llocs lliures en totes les edicions de cursos de c

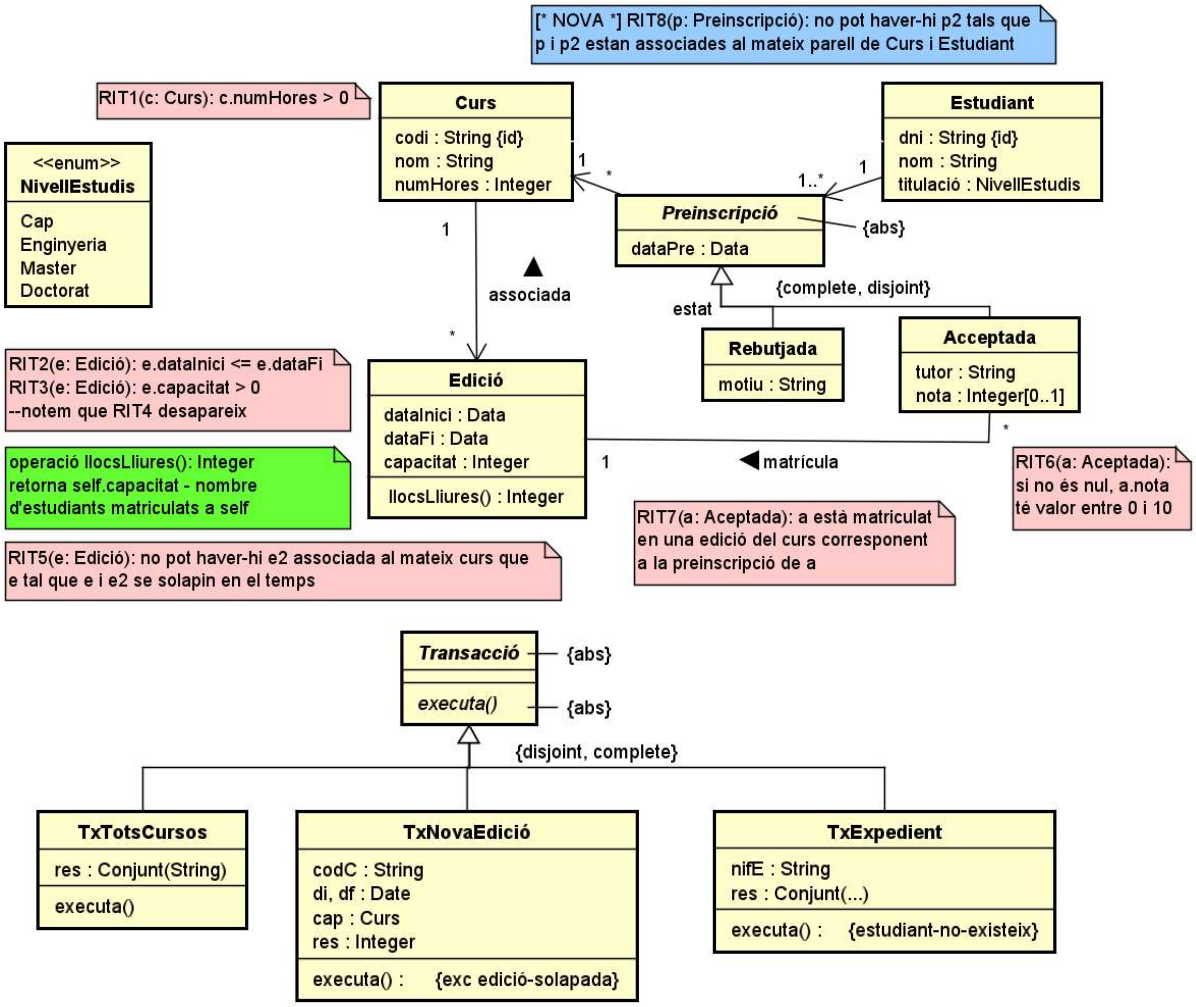
1.2

operació CapaDeDomini::totsCursos(): Conjunt(codC: String)
pre hi ha com a mínim un curs en el sistema – ho diu l'enunciat
retorna el conjunt dels codis dels cursos existents

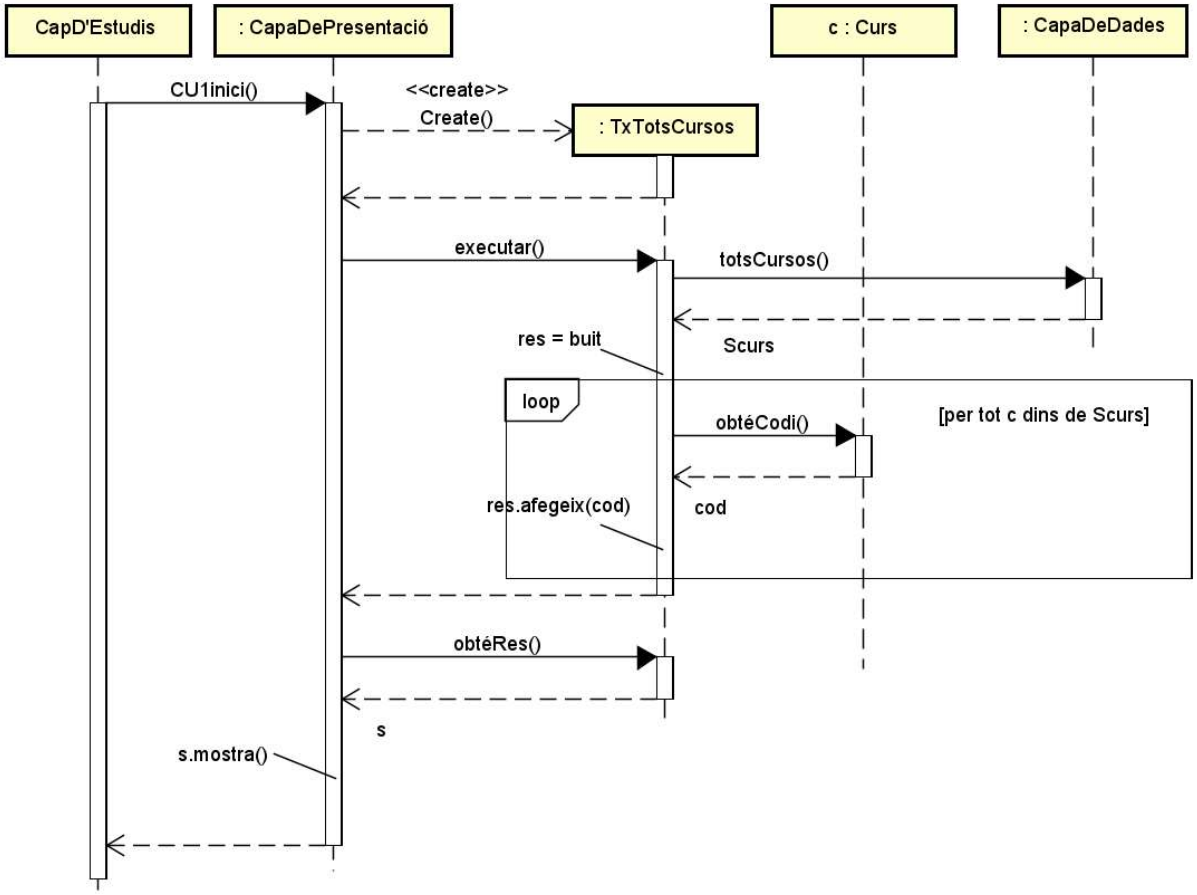
operació CapaDeDomini::novaEdició(codC: String; di, df: Data; cap: Integer): Integer
pre existeix un curs c amb c.codi = codC – *atès que el codi se selecciona d'un desplegable*
pre di <= df -- *RIT2, controlada per la capa de presentació, segons diu l'enunciat de l'apartat 1*
pre cap > 0 -- *RIT3, controlada per la capa de presentació, segons diu l'enunciat de l'apartat 1*
exc *edició-solapada*: existeix una edició e2 tal que l'interval [e2.dataInici, e2.dataFi] se solapa amb [di, df] -- *RIT5; la capa de presentació no la controla*
post crea una Edició e = (di, df, cap) i l'associa amb c
retorna el nombre de llocs lliures en totes les edicions de cursos de c

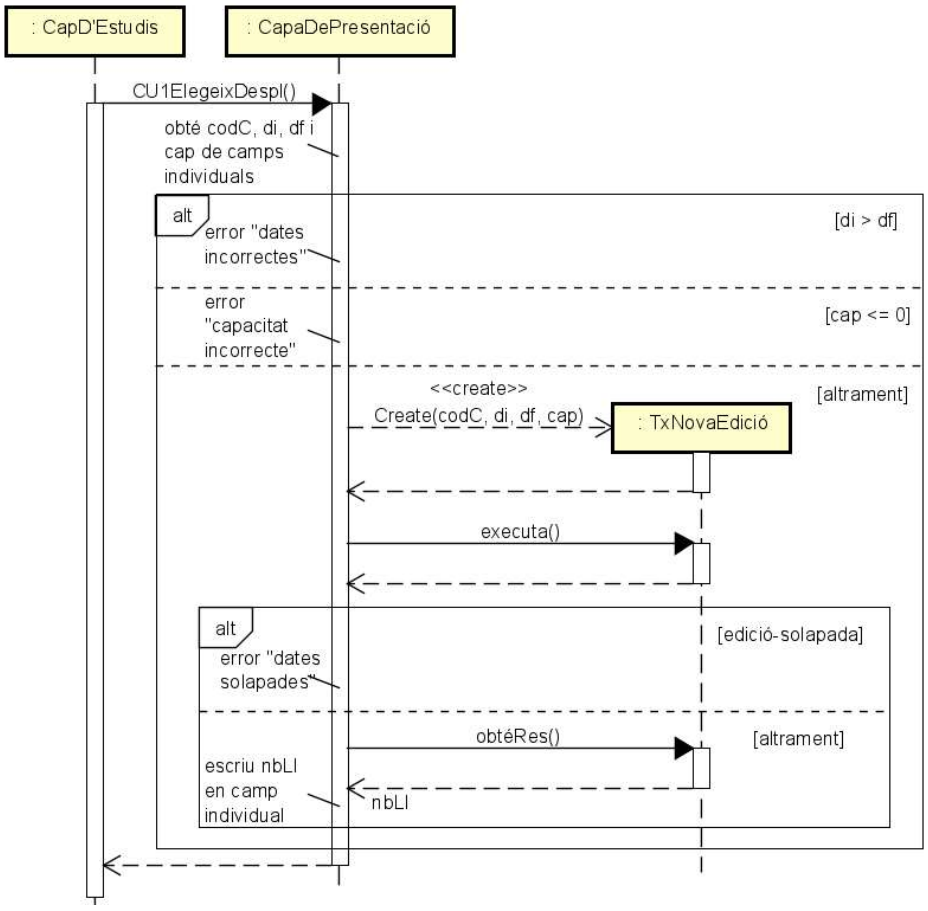
operació CapaDeDomini::expedient(dniE: String): Conjunt(codC: String+dP: Data+not: Integer)
exc *estudiant-no-existeix*: no existeix cap estudiant e amb e.dni = dniE
retorna un conjunt definit segons l'especificació donada a l'enunciat

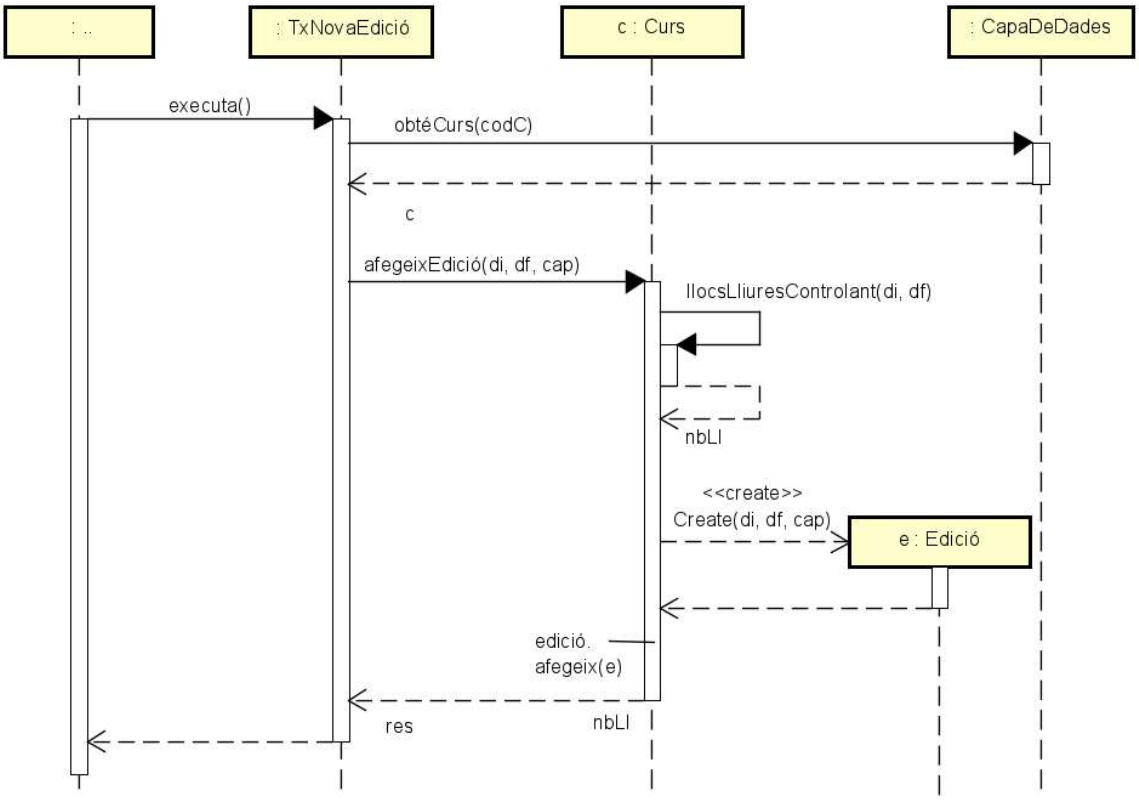
1.3 (Nota preliminar: aquest és un problema per aplicar Domain Model i Data Mapper. L'ús de Passarel.les i Cercadors està fortament penalitzat, així com fer diagrames de seqüència sense haver simplificat el model de dades prèviament.)



Nota: per simplicitat dels diagrames, no es mostren explícitament les etiquetes "singleton" (tampoc les hem considerat a les vostres solucions).

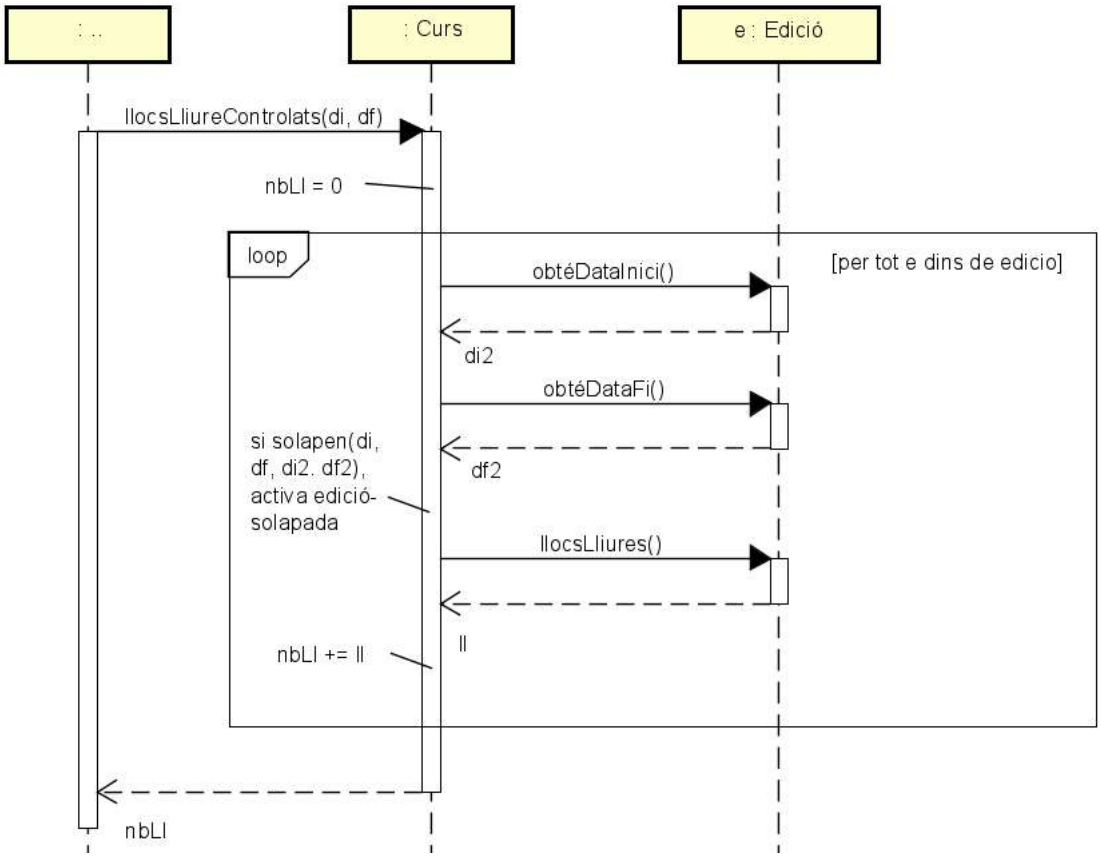

CapD'Estudis
: CapaDePresentació
c : Curs
: CapaDeDades
CU1inici()
<<create>>
Create()
: TxTotsCursos
executar()
totsCursos()
Scurs
res = buit
loop
[per tot c dins de Scurs]
obtéCodi()
cod
res.afegeix(cod)
obtéRes()
s
s.mostra()


: CapD'Estudis
: CapaDePresentació
CU1ElegeixDespl()
obté codC, di, df i cap de camps individuals
alt
error "dates incorrectes"
[di > df]
error "capacitat incorrecte"
[cap <= 0]
<<create>>
Create(codC, di, df, cap)
: TxNovaEdició
[altrament]
executa()
alt
error "dates solapades"
[edició-solapada]
obtéRes()
[altrament]
escriu nbLI en camp individual
nbLI

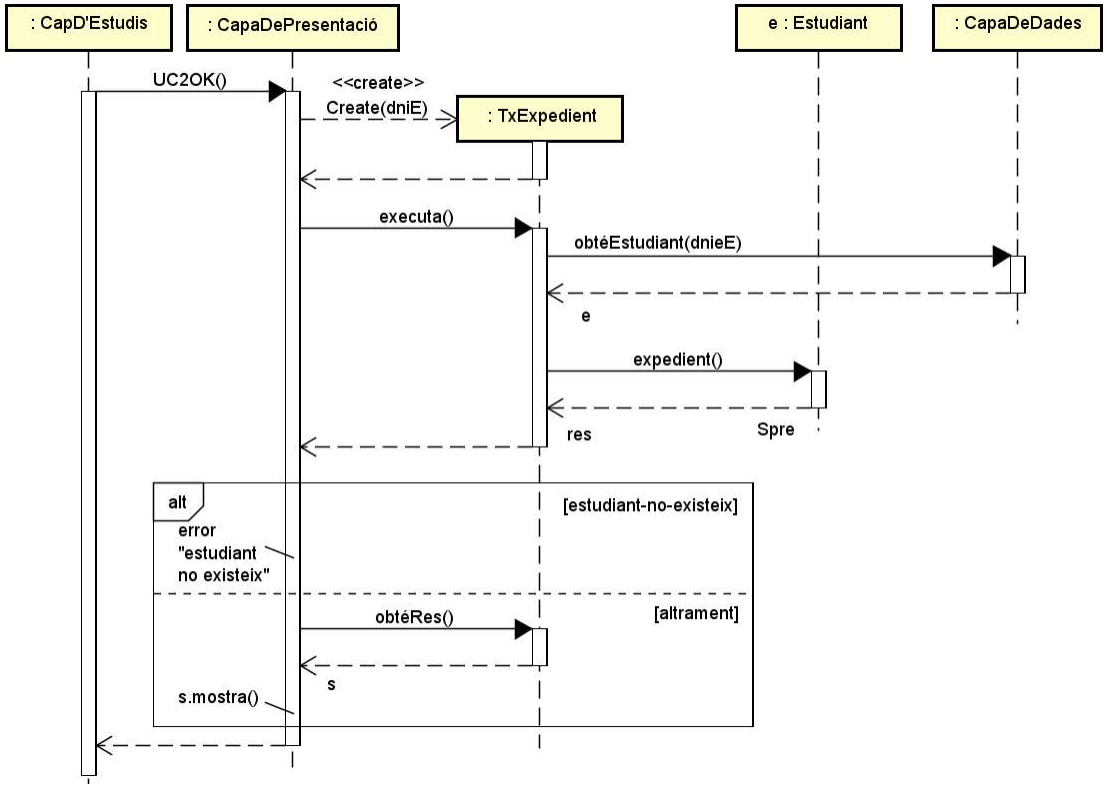


operació Curs::afegeixEdició(di: Date; df: Date; cap: Integer)
pre di <= df
pre cap > 0
exc edició-solapada: hi ha una e2 associada a self tal que e2 se solapa en el temps amb (di, df)
post crea una Edició e = (di, df, cap) i l'associa amb self

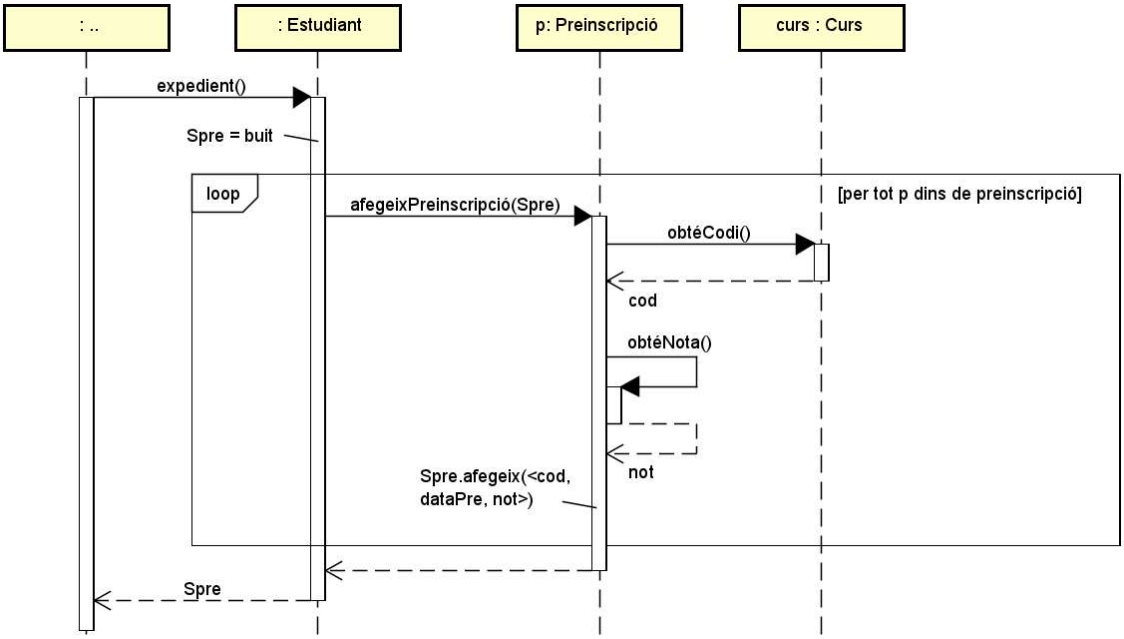
operació Curs::llocsLliuresControlant(di: Date; df: Date): Integer
– nota: s'ha optat per una única operació que a la vegada computi els llocs lliures i controli solapament, per millorar l'eficiència**.** Una altra possibilitat igualment vàlida és tenir una operació diferent per cada cosa, en aquest cas afavorint la cohesió de la solució
pre di <= df
exc edició-solapada: hi ha una e2 associada a self tal que e2 se solapa en el temps amb (di, df)
retorna nombre de llocs lliures en totes les edicions de cursos de self



operació Edició::llocsLliures(): Integer
retorna nombre de llocs lliures de self



operació Estudiant::expedient(): Conjunt(codC: String+dP: Data+not: Integer)
retorna l'expedient de self segons l'especificació donada a l'enunciat

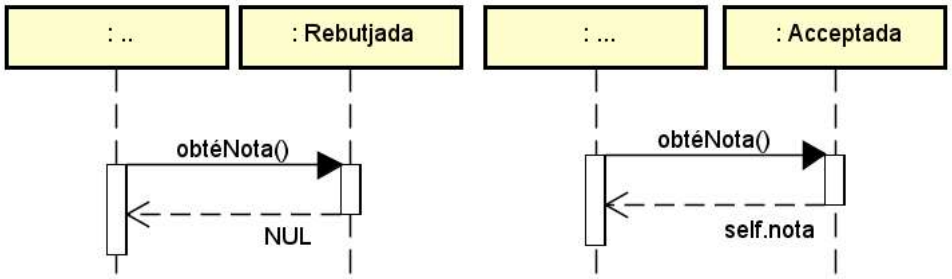


operació Preinscripció::afegeixPreinscripció(**in/out** Spre: Conjunt(codC: String+dP: Data+not: Integer)
post afegeix a Spre l'expedient associat a self segons l'especificació donada a l'enunciat

operació Preinscripció::obtéNota(): Integer {abs} – patró Plantilla; apliquem polimorfisme
retorna NUL si self és Rebutjada, i self.nota si self és Acceptada

operació Rebutjada::obtéNota(): Integer
retorna NUL

operació Acceptada::obtéNota(): Integer
retorna self.nota



Problema 2: Disseny amb Transaction Script i Row Data Gateway [4 punts].

Sigui el cas d'ús essencial del model de l'especificació següent:

CU3. *Gestionar Preinscripció*. El Cap d'Estudis rep una petició de preinscripció d'un estudiant (que ha d'existir en el sistema) en un curs i, atès el seu nivell d'estudis, decideix si accepta la preinscripció o la rebutja, en aquest cas indicant el motiu. Si l'accepta, li assigna un tutor i el matrícula en una edició del curs donant les seves dates d'inici i fi. El Sistema enregistra la petició i en cas d'acceptació, mostra la suma del número d'hores dels cursos on l'estudiant té preinscripció acceptada i encara no té nota.

Suposeu que el disseny de la interfície d'usuari no demana cap canvi en aquest model del comportament (no hi ha desplegables ni res per l'estil).

Apartat 1 [1 punts]. Dissenyeu l'esquema de la base de dades relacional corresponent al model conceptual presentat al full adjunt. Mostreu clarament quines restriccions d'integritat (textuals i gràfiques) queden sota el control de l'esquema. Suposeu ara que l'atribut derivat "/llocsLliures" es materialitza.

Apartat 2 [2.5 punts]. Dissenyeu el diagrama de seqüència de les operacions de la capa de domini corresponent a **CU3**. En aquest problema, **no** cal dissenyar les operacions de la capa de presentació. Useu un controlador cas d'ús per la capa de domini. **No** cal escriure els contractes de les operacions. Mostreu les excepcions que es poden activar en les operacions de la capa de dades mitjançant el comentari "exc *nom de l'excepció*" en el diagrama de seqüència (només d'aquelles que realment es poden activar en aquest problema).

Apartat 3 [0.5 punts]. Feu el diagrama de classes de disseny de la capa de domini. Heu d'incloure totes les classes, amb els atributs i operacions (tret de les creadores ni els *getters* i *setters*). No oblideu de declarar les excepcions associades a les operacions.

Solució Exercici 2

2.1. Disseny de la base de dades

NivellEstudis(<u>nivell</u>)

- nivell: CHECK (nivell IN (‘Cap’, ‘Enginyeria’, ‘Master’, ‘Doctorat’)) – podria ser directament un valor a la taula d’Estudiant

Estudiant(<u>dni</u>, nom, titulació)

- titulació: REFERENCIA (NivellEstudis)

Curs(<u>codi</u>, nom, numHores)

- numHores: CHECK (numHores > 0) -- RIT1

Edició(<u>codiC, dataInici, dataFi</u>, capacitat, llocsLliures)

- codiC: REFERENCIA (Curs)
- dataInici+dataFi: CHECK (dataInici <= dataFi) -- RIT2
- capacitat: CHECK (capacitat > 0) -- RIT3
- llocsLliures: CHECK (llocsLliures >= 0) -- RIT4

Preinscripció(<u>codiC, dniE</u>, dataPre, estat) –estat no és obligatori però facilita el disseny

- codiC: REFERENCIA (Curs)
- dniE: REFERENCIA (Estudiant)
- estat: CHECK (estat IN (‘rebutjada’, ‘acceptada’)

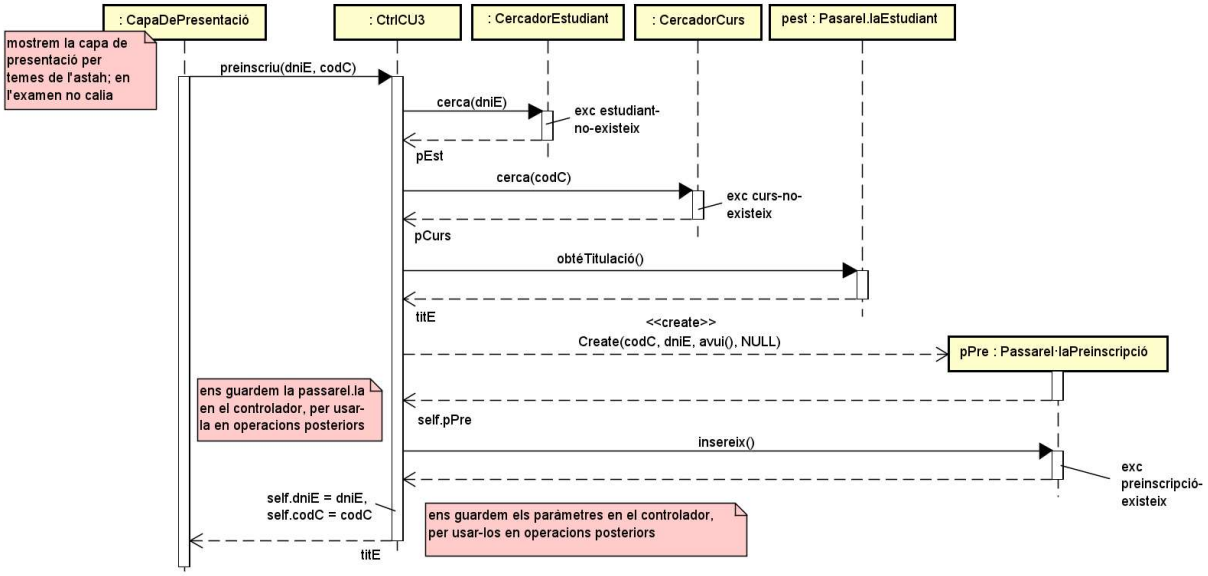
Rebutjada(<u>codiC, dniE</u>, motiu)

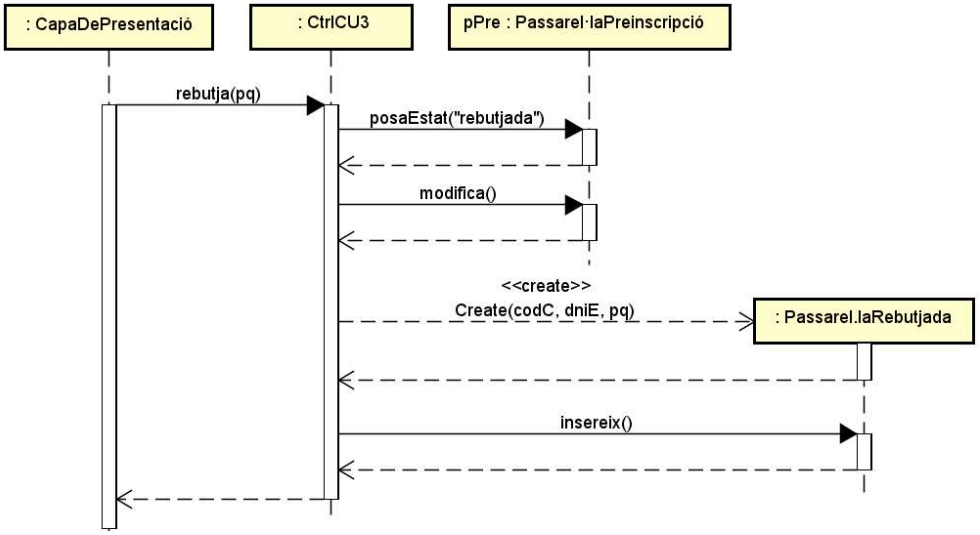
- codiC+dniE: REFERENCIA (Preinscripció)
- codiC: REFERENCIA (Curs)
- dniE: REFERENCIA (Estudiant)

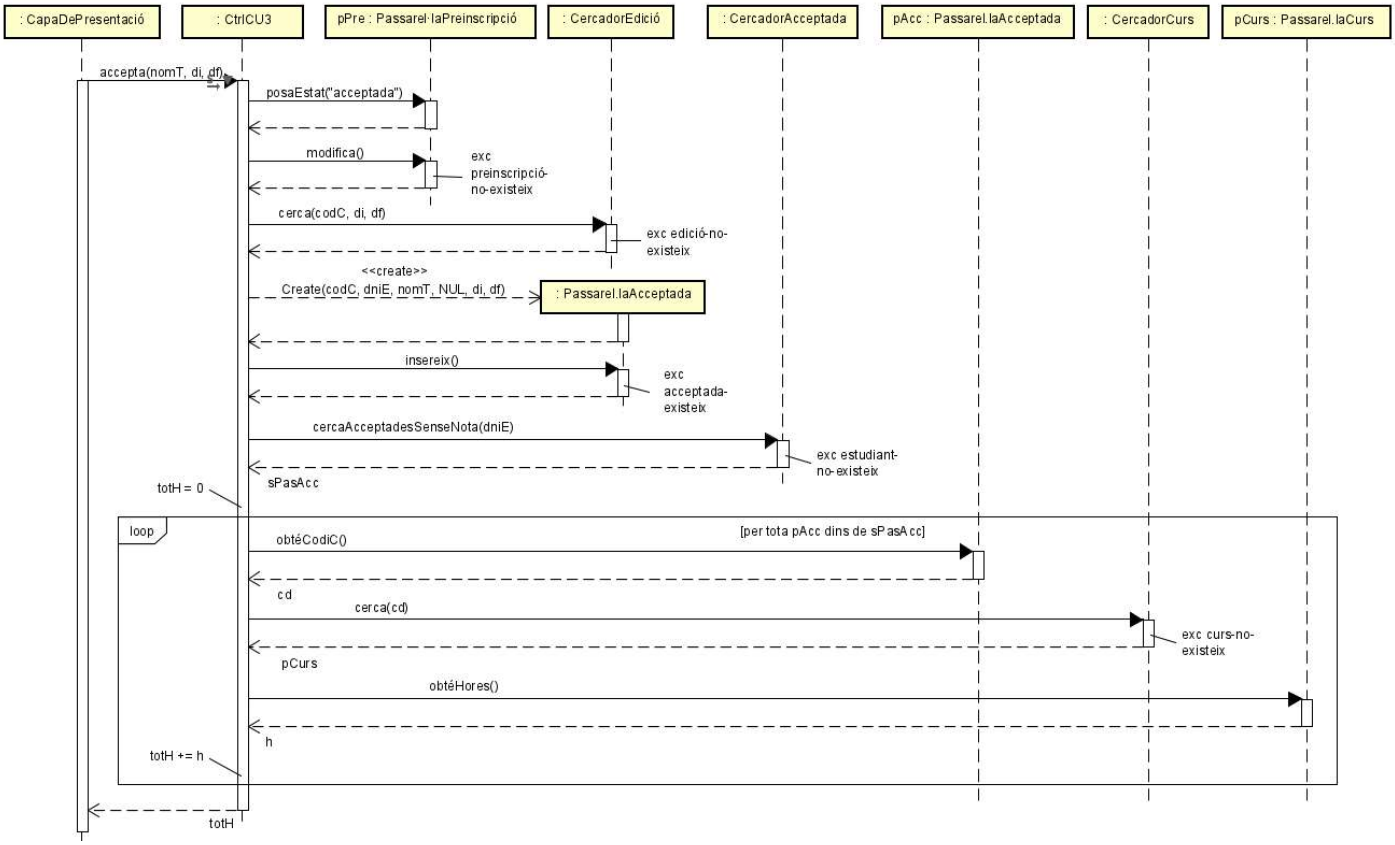
Acceptada(<u>codiC, dniE</u>, tutor, nota, dataInici, dataFi)

- codiC+dniE: REFERENCIA (Preinscripció)
- codiC: REFERENCIA (Curs)
- dniE: REFERENCIA (Estudiant)
- nota: CHECK (nota = NULL OR (nota >=0 AND nota <=10)) -- RIT6
- codiC+dataInici+dataFi: REFERENCIA (Edició)

2.2. Disseny diagrames de seqüència



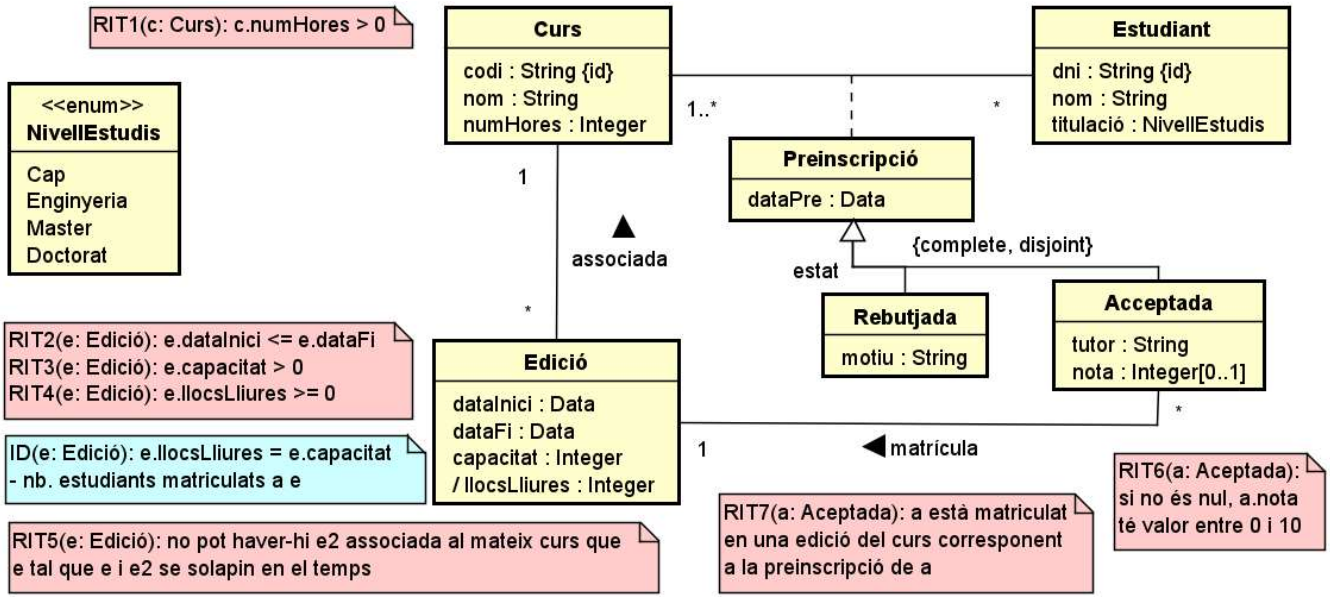




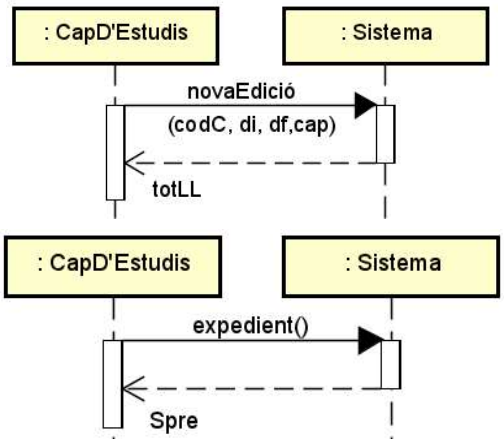
2.3 Capa de domini

CtrlCU3
pPre : Passarel·laPreinscripció codC : String dniE : String
preinscriu(dniE : String, codC : String) : String {exc estudiant-no-existeix, curs-no-existeix, preinscripció-existeix} rebutja(pq : String) accepta(nomT : String, di : Date, df : Date) : Integer {exc edició-no-existeix}

FULL ADJUNT: ESPECIFICACIÓ DE PARTIDA



Problema 1

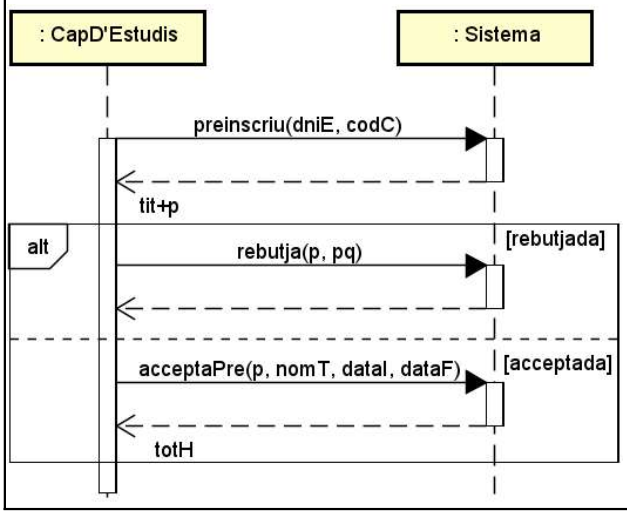


operació novaEdició(codC: String; di, df: Data; cap: Integer): Integer
pre existeix Curs c amb c.codi = codC
post crea una Edició e = (di, df, cap) i l'associa amb c
retorna nombre de llocs lliures en totes les edicions de cursos de c

operació expedient(dniE: String): Cjt(codC: String+dP: Data+not: Int)
pre existeix Estudiant e amb e.dni = dniE
retorna un conjunt que conté una tripleta de valors (codC, dP, not) per cada p preinscripció d'e, tals que:

- codC és el codi del curs de p
- dP = p.dataPre
- si p.estat = Rebutjada, aleshores not = NUL
- si p.estat = Acceptada, aleshores not = p.nota

Problema 2



operació preinscriu(dniE: String; codC: String): NivellEstudis + Preinscripció
pre existeix Estudiant e = (dniE, nomE, titE)
pre existeix Curs c amb c.codi = codC
post crea Preinscripció p tal que p.dataPre = avui() i p está associada amb e i c
retorna titE + p

operació rebutja(p: Preinscripció; pq: String)
post p.estat = Rebutjada, p.motiu = pq

operació accepta(p: Preinscripció; nomT: String; di, df: Data): Integer
pre existeix Edició e del curs de p amb e.dataInici = di, e.dataFi = df
post p.estat = Acceptada, p.tutor = nomT, p.nota = NUL
post p està matriculat a e
retorna la suma del número d'hores dels cursos on l'estudiant té preinscripció acceptada i encara no té nota